Руководство по разработке Eiros Agent

Рекомендации GitHub Copilot - Реализованные исправления

1. Устранение устаревших iOS API ✅

Проблема: Использование `UIScreen.main.bounds.width` в ControlRoomView.swift
Решение: Заменено на современный подход с `GeometryReader`

```
// ❌ Устаревший подход (iOS 13+)
.frame(maxWidth: UIScreen.main.bounds.width * 0.75)

// ✅ Современный подход
GeometryReader { geometry in
    // ...
    .frame(maxWidth: geometry.size.width * 0.75)
}
```

2. Устранение дублирования кода ✅

Проблема: Дублирование между ControlRoomView и EnhancedControlRoomView, LLMManager и EnhancedLLMManager

Решение: Создана архитектура с протоколами и базовыми классами:
- `LLMManagerProtocol` - общий интерфейс для всех LLM менеджеров
- `BaseLLMManager` - базовый класс с общей функциональностью
- `MessageBubbleView` - переиспользуемый компонент для сообщений

3. Улучшение архитектуры ✅

Новые компоненты:
- `EirosAgent/Protocols/LLMManagerProtocol.swift` - протоколы для стандартизации
- `EirosAgent/Managers/BaseLLMManager.swift` - базовая функциональность
- `EirosAgent/Views/Components/MessageBubbleView.swift` - современный UI компонент

Рекомендации для будущих PR

Документирование Mock-реализаций

При создании mock-реализаций (как в случае с Qwen2VL), следуйте этим рекомендациям:

1. Четкая документация в docstring

```
/**
 * Qwen2VL Analyzer - Advanced multimodal vision-language analysis
 *
 * ⚠️ IMPORTANT: This is currently a MOCK IMPLEMENTATION
 *
 * Current Status:
 * - ✅ API structure and interfaces are production-ready
 * - ❌ Actual Qwen2VL model integration is not implemented
 * - ❌ Returns mock responses for development/testing purposes
 *
 * Production Readiness:
 * - To use in production, implement actual API calls in `_analyze_with_api()`
 * - Implement local model loading in `_analyze_with_local_model()`
 * - Add proper error handling for real API responses
 *
 * Mock Mode Indicators:
 * - Check `isMockMode` property to determine current state
 * - Mock responses include "Mock" prefix in analysis results
 * - All mock methods log "mock implementation" warnings
 */
class Qwen2VLAnalyzer {
    /// Indicates whether this instance is running in mock mode
    public let isMockMode: Bool = true

    // ... rest of implementation
}
```

2. Явные индикаторы mock-режима

```
func analyze_with_api(image_b64: String, prompt: String) -> Dict[String, Any] {
    if isMockMode {
        logger.warning("🚧 MOCK MODE: analyze_with_api returning mock response")
        return createMockResponse(prompt: prompt)
    }

    // Real implementation would go here
    return performActualAPICall(image_b64, prompt)
}
```

3. Конфигурационные флаги

```
struct Qwen2VLConfig {
    let enableMockMode: Bool
    let apiEndpoint: String?
    let apiKey: String?

    var isProductionReady: Bool {
        return !enableMockMode && apiEndpoint != nil && apiKey != nil
    }
}
```

Использование абсолютных импортов

```
// ❌ Относительные импорты
from ..core.exceptions import VisionError

// ✅ Абсолютные импорты
from eiros_agent.core.exceptions import VisionError
```

Стандарты кодирования

Swift

- Используйте `GeometryReader` вместо `UIScreen.main`
- Применяйте протоколы для уменьшения дублирования
- Создавайте переиспользуемые компоненты в папке `Components/`

Python

- Используйте абсолютные импорты
- Документируйте mock-реализации
- Добавляйте type hints для всех публичных методов

Архитектурные принципы

1. Принцип единственной ответственности

Каждый класс должен иметь одну причину для изменения.

2. Принцип открытости/закрытости

Классы должны быть открыты для расширения, но закрыты для модификации.

3. Принцип подстановки Лисков

Объекты должны быть заменяемы экземплярами их подтипов.

4. Принцип разделения интерфейса

Клиенты не должны зависеть от интерфейсов, которые они не используют.

5. Принцип инверсии зависимостей

Зависимости должны быть направлены на абстракции, а не на конкретные реализации.

Контрольный список для PR

- [ ] Нет использования устаревших iOS API
- [ ] Mock-реализации четко документированы
- [ ] Используются абсолютные импорты
- [ ] Нет дублирования кода
- [ ] Добавлены unit тесты
- [ ] Обновлена документация
- [ ] Проверена совместимость с iOS 14+

Инструменты для проверки качества кода

Swift

```
# SwiftLint для проверки стиля кода
swiftlint lint

# SwiftFormat для автоматического форматирования
swiftformat .
```

Python

```
# Black для форматирования
black .

# Flake8 для проверки стиля
flake8 .

# MyPy для проверки типов
mypy .
```

Примечание: Этот документ будет обновляться по мере развития проекта и получения новых рекомендаций от GitHub Copilot.